Resumo das Escolhas Arquiteturais

Para o desenvolvimento do sistema, foi escolhida uma arquitetura baseada em **Monólito Modular utilizando Arquitetura Hexagonal**, implementada com **NestJS e TypeScript**. A solução também utilizará o padrão **MVC na camada de entrada HTTP** e integrações com os sistemas externos **iEducar** e **Moodle**.

A escolha do **Monólito Modular** para a primeira etapa do projeto tem como principal objetivo manter a simplicidade de desenvolvimento, implantação e manutenção, sem abrir mão de uma separação clara entre as responsabilidades do sistema. Diferentemente de um monólito tradicional, no qual diferentes funcionalidades podem acabar fortemente acopladas, o monólito modular divide a aplicação em módulos de negócio bem definidos, como acadêmico, sincronização, relatórios e autenticação.

Essa organização também foi escolhida considerando a segunda etapa do trabalho, na qual a solução deverá evoluir para uma arquitetura de microsserviços. Ao definir desde o início limites claros entre os módulos, torna-se mais simples extrair determinadas funcionalidades do monólito e transformá-las posteriormente em serviços independentes.

A **Arquitetura Hexagonal**, também conhecida como Ports and Adapters, foi escolhida principalmente devido à forte dependência do sistema de integrações externas, especialmente com o iEducar e o Moodle. O objetivo é evitar que as regras de negócio da aplicação dependam diretamente dessas plataformas.

Na arquitetura hexagonal, as regras e os casos de uso ficam no núcleo da aplicação, enquanto tecnologias externas ficam localizadas nas bordas. A comunicação entre essas partes ocorre através de interfaces, chamadas de *ports*, que são implementadas por *adapters*.

Por exemplo, a aplicação pode definir uma porta genérica para representar uma plataforma virtual de aprendizagem, sem conhecer especificamente o Moodle. Um adapter do Moodle implementa essa interface e é responsável por realizar as chamadas para a API externa. O mesmo princípio é utilizado na integração com o iEducar.

Essa abordagem reduz o acoplamento entre a aplicação e os sistemas externos. Dessa forma, caso uma integração seja modificada ou até mesmo substituída no futuro, o impacto sobre as regras de negócio tende a ser menor.

Também será aplicada uma **Anti-Corruption Layer (ACL)** nas integrações. Isso significa que os modelos de dados específicos do Moodle e do iEducar não serão utilizados diretamente pelo domínio da aplicação. Os adapters serão responsáveis por converter os dados externos para modelos internos do sistema. Assim, detalhes específicos das APIs externas não se espalham pelo restante do código.

O padrão **MVC não foi descartado**. Ele será utilizado principalmente na camada HTTP da aplicação. Os Controllers do NestJS atuarão como ponto de entrada das requisições, recebendo os dados enviados pelos usuários e chamando os casos de uso da aplicação.

Dessa maneira, MVC e Arquitetura Hexagonal são utilizados em níveis diferentes e complementares: o MVC organiza a interação HTTP, enquanto a Arquitetura Hexagonal organiza a aplicação e suas dependências.

O **NestJS** foi escolhido como framework por possuir uma estrutura fortemente baseada em módulos e injeção de dependência, características que facilitam a implementação de um monólito modular e da Arquitetura Hexagonal. Sua organização também favorece uma futura separação em microsserviços, já que módulos independentes podem ser extraídos com menor impacto sobre o restante do sistema.

O **TypeScript** foi escolhido por oferecer tipagem estática sobre JavaScript, melhorando a segurança durante o desenvolvimento, facilitando refatorações e tornando as interfaces entre módulos e adapters mais explícitas. Isso é especialmente relevante em uma arquitetura baseada em ports e adapters, pois contratos entre componentes podem ser definidos através de interfaces.

A aplicação utilizará um banco de dados próprio, preferencialmente **PostgreSQL**. Entretanto, esse banco não deverá duplicar integralmente os dados armazenados pelo iEducar ou Moodle. Cada sistema terá responsabilidades bem definidas sobre suas informações.

O **iEducar será considerado a principal fonte de verdade dos dados acadêmicos**, como alunos, professores, turmas, disciplinas e matrículas. O **Moodle será responsável pelos dados relacionados ao ambiente virtual de aprendizagem**, como cursos virtuais, atividades, conteúdos e participação dos alunos. O sistema desenvolvido ficará responsável principalmente pelas informações necessárias para realizar e controlar a integração entre as duas plataformas, como mapeamentos de identificadores, histórico de sincronizações, erros e auditoria.

Um possível fluxo de funcionamento será a identificação de uma matrícula realizada no iEducar, seguida pela sincronização dessa informação com o Moodle. O sistema consulta os dados acadêmicos através de um adapter do iEducar, converte esses dados para o modelo interno da aplicação e, em seguida, utiliza um adapter do Moodle para realizar a operação correspondente no ambiente virtual.

A sincronização poderá ser executada periodicamente por meio de um processo agendado dentro do próprio monólito. O sistema deverá registrar o resultado de cada sincronização, incluindo operações concluídas, falhas e tentativas de reprocessamento.

Para a primeira etapa do trabalho, não será utilizada uma arquitetura de microsserviços porque ela adicionaria complexidades desnecessárias para o estágio inicial do projeto, como comunicação entre serviços, múltiplos bancos de dados, descoberta de serviços, observabilidade distribuída, tratamento de falhas de rede, mensageria e consistência eventual.

O monólito modular oferece uma solução mais simples para o desenvolvimento inicial, mantendo ao mesmo tempo uma estrutura preparada para evolução.

Na segunda etapa do projeto, os módulos definidos no monólito poderão ser analisados e transformados em **microsserviços independentes**. Por exemplo, funcionalidades relacionadas à sincronização, dados acadêmicos ou relatórios poderão ser separadas em serviços distintos. A comunicação entre esses serviços poderá ocorrer através de APIs HTTP ou de mensageria, utilizando tecnologias como RabbitMQ ou Kafka, dependendo das necessidades identificadas.

Portanto, a arquitetura adotada para a primeira etapa pode ser resumida como:

NestJS + TypeScript + Monólito Modular + Arquitetura Hexagonal + MVC na camada HTTP + PostgreSQL + Adapters para iEducar e Moodle.

Essa combinação foi escolhida por oferecer simplicidade inicial, baixo acoplamento com sistemas externos, separação clara de responsabilidades, facilidade de manutenção e uma base adequada para a posterior evolução da aplicação para uma arquitetura de microsserviços.